Crio.Do

LEARNING ROADMAP

# Essential Developer Tools

The Tools That Actually Make You Faster

| Intensive | Balanced | Flexible |
| --- | --- | --- |
| 15-20 hrs/week | 5-8 hrs/week | 2-4 hrs/week |

▪ 7 Phases ★ Beginner to Intermediate

https://crio.do

# Overview

This isn't a list of 50 tools you'll never use. It's the concentrated set of tools and techniques that compound your productivity over years. Senior developers didn't get faster by learning more languages — they got faster by mastering their tools. A developer who knows their terminal, Git, debugger, and IDE shortcuts will outperform someone with twice the 'experience' who doesn't. These skills transfer across every language, framework, and job.

### Who is this for?

Any developer who wants to stop fumbling with tools and start flowing. Especially valuable for developers in their first 1-3 years who want to build habits that compound, or experienced developers who never properly learned their tools.

### Prerequisites

- Basic programming experience in any language
- A computer (Mac, Linux, or Windows with WSL)
- Willingness to feel slow before getting fast
- Patience to build muscle memory

### What you'll learn

- Navigate and manipulate files from the terminal without thinking
- Use Git confidently — including rebase, stash, and recovering from mistakes
- Debug efficiently using proper tools, not just print statements
- Code faster with keyboard shortcuts and IDE features
- Test APIs and understand HTTP at a practical level
- Use containers for consistent development environments
- Leverage AI assistants without becoming dependent on them

# Learning Plans

Choose a plan that fits your schedule. All plans cover the same content.

## Intensive

Fast Track

**15-20** hrs/week

**60-80** total hrs

Focused practice alongside your regular work. Best learned by using these tools on real projects, not in isolation.

Ideal for:

- Developers between jobs
- Those with dedicated learning time
- Anyone frustrated with their current speed

## Balanced

Recommended

**5-8** hrs/week

**60-100** total hrs

Learn one tool deeply each week while working. The recommended approach — real projects provide the best practice.

Ideal for:

- Working developers
- Students with projects
- Most people

## Flexible

Self-paced

**2-4** hrs/week

**80-120** total hrs

Gradual skill building. Focus on one new shortcut or command per day until it becomes automatic.

Ideal for:

- Busy professionals
- Those who prefer slow habit building
- Weekend learners

# Phase Schedule by Plan

| Phase | Topic | Intensive | Balanced | Flexible |
|---|---|---|---|---|
| 1 | Terminal & Shell Mastery | Week 1 | Weeks 1-2 | Weeks 1-4 |
| 2 | Git Beyond Basics | Week 2 | Weeks 3-4 | Weeks 5-8 |
| 3 | IDE & Editor Mastery | Week 3 | Weeks 5-6 | Weeks 9-12 |
| 4 | HTTP & API Tools | Week 4 | Weeks 7-8 | Weeks 13-16 |
| 5 | Browser DevTools Deep Dive | Week 5 | Weeks 9-10 | Weeks 17-20 |
| 6 | Docker Essentials | Week 6 | Weeks 11-12 | Weeks 21-26 |
| 7 | AI Coding Assistants | Week 7 | Weeks 13-14 | Weeks 27-30 |

# 1 Terminal & Shell Mastery

Intensive: Week 1 | Balanced: Weeks 1-2 | Flexible: Weeks 1-4

The terminal is the great equalizer. It works the same whether you're on a MacBook or SSH'd into a server in Singapore. Developers who avoid the terminal hit a ceiling; those who embrace it unlock automation, scripting, and a level of control that GUIs can't match. This isn't about memorizing commands — it's about building fluency.

## Learning Objectives

- Navigate directories and manipulate files without hesitation
- Understand pipes, redirection, and command composition
- Use modern CLI tools that are 10x better than defaults
- Write simple shell scripts to automate repetitive tasks
- Customize your shell for productivity

## Topics to Cover

### Navigation & File Operations

The basics you use 100 times a day. Speed here compounds massively.

- cd, ls, pwd (with useful flags)
- mkdir, rm, mv, cp
- touch, cat, less, head, tail
- find (basic patterns)
- Absolute vs relative paths
- Tab completion (use it religiously)

### Pipes & Redirection

WHY: This is the Unix philosophy — small tools that do one thing well, combined. Once you understand pipes, you can build complex operations from simple parts.

- | (pipe output to next command)
- > and >> (write/append to file)
- < (read from file)
- 2>&1 (redirect stderr)
- xargs (build commands from input)
- tee (write to file AND stdout)

### Modern CLI Tools

WHY: Default Unix tools are decades old. Modern alternatives are faster, prettier, and more intuitive. These are quality-of-life improvements that add up.

- fzf — fuzzy finder for everything
- ripgrep (rg) — faster grep with sane defaults
- bat — cat with syntax highlighting
- exa/eza — modern ls with git integration
- jq — manipulate JSON like a pro
- tldr — simplified man pages

### Shell Customization

WHY: Your shell is your home. A good prompt shows useful info. Aliases save keystrokes. Invest 2 hours here, save hundreds over your career.

- Zsh + Oh My Zsh (or Fish)
- Useful aliases (~/.zshrc)
- Prompt customization (Starship)
- History search (Ctrl+R)
- Environment variables
- PATH management

## Hands-on Projects

### Log File Analyzer

Write a one-liner that: finds all .log files in a directory, extracts lines containing 'ERROR', counts occurrences by error type, and sorts by frequency. Use pipes to chain grep, cut, sort, and uniq.

Pipes | Text processing | Command chaining

### Directory Setup Script

Write a bash script that sets up your standard project structure: creates directories, initializes git, creates boilerplate files, and opens your editor. Use it for every new project.

Bash scripting | Automation | Variables

## Resources

COURSE **The Missing Semester (MIT)**

TOOL **explainshell.com**

RESOURCE **Modern Unix Tools List**

### Milestone

Complete common tasks without touching the mouse. You should be able to navigate to any directory, find files, search content, and chain commands together. Install and configure at least fzf and ripgrep.

# 2 Git Beyond Basics

Intensive: Week 2 | Balanced: Weeks 3-4 | Flexible: Weeks 5-8

Everyone knows git add, commit, push. But most developers panic when something goes wrong because they don't understand Git's model. This phase takes you from 'I know the commands' to 'I understand what's happening.' You'll learn workflows that keep history clean, recover from mistakes confidently, and collaborate without merge hell.

## Learning Objectives

- Understand Git's data model (commits, branches, refs)
- Use rebase to maintain clean history
- Recover from almost any mistake
- Work with stash effectively
- Find bugs efficiently with bisect
- Write meaningful commit messages

## Topics to Cover

### Git's Mental Model

WHY: Git commands make sense once you understand that Git is a directed acyclic graph of snapshots. Without this mental model, Git is just magic incantations.

- Commits are snapshots, not diffs
- Branches are just pointers
- HEAD and refs explained
- The three trees (working, staging, repo)
- git log --graph visualization
- Understanding detached HEAD

### Rebase & Clean History

WHY: Clean history is readable history. Rebase lets you tell a coherent story instead of 'fixed typo' → 'actually fixed it' → 'merge conflict resolved'. Teams with clean git history onboard faster and debug easier.

- git rebase vs git merge
- Interactive rebase (squash, edit, reorder)
- When to rebase vs merge
- Rebase golden rule (never rebase public branches)
- git pull --rebase
- Cleaning up before PR

### Recovery & Safety Nets

WHY: Fear of breaking things slows you down. Once you know you can recover from anything, you experiment freely. The reflog is your time machine.

- git reflog (your safety net)
- git reset (soft, mixed, hard)
- git revert (safe undo)
- git checkout -- file (discard changes)
- Recovering deleted branches
- git stash (and stash pop, list, apply)

### Advanced Workflows

WHY: These commands solve specific problems elegantly. Cherry-pick saves you from rewriting code. Bisect finds bugs in minutes instead of hours.

- git cherry-pick (apply specific commits)
- git bisect (binary search for bugs)
- git blame (who wrote this?)
- git worktree (multiple checkouts)
- .gitignore patterns
- Git hooks basics

## Hands-on Projects

### Interactive Rebase Practice

Create a branch with 5 messy commits ('wip', 'fix', 'oops', etc.). Use interactive rebase to squash them into 2-3 meaningful commits with good messages. Then practice recovering the original commits using reflog.

Interactive rebase | Squashing | Reflog recovery

### Git Bisect Bug Hunt

Clone a repository with a known bug introduced somewhere in history. Use git bisect to find the exact commit that introduced it. Practice writing good/bad scripts for automated bisect.

git bisect | Binary search debugging | Automation

## Resources

BOOK **Pro Git Book (Free)**

INTERACTIVE **Learn Git Branching (Interactive)**

REFERENCE **Oh Shit, Git!?!**

TOOL **LazyGit (Terminal UI)**

### Milestone

Rebase a feature branch onto main without fear. Recover a 'lost' commit using reflog. Use bisect to find a bug. Your commits should tell a story, not a diary of your debugging session.

# 3 IDE & Editor Mastery

Intensive: Week 3 | Balanced: Weeks 5-6 | Flexible: Weeks 9-12

Your editor is where you spend most of your time. A developer who knows their shortcuts codes at the speed of thought. One who doesn't constantly breaks flow to reach for the mouse. This isn't about memorizing 200 shortcuts — it's about the 20 that matter most and building them into muscle memory.

## Learning Objectives

- Navigate code without touching the mouse
- Use multi-cursor editing effectively
- Master find/replace including regex
- Utilize refactoring tools (rename, extract)
- Debug using IDE debugger, not print statements
- Customize your editor for your workflow

## Topics to Cover

### The 20 Shortcuts That Matter

WHY: Keyboard shortcuts aren't about speed — they're about staying in flow. Each mouse reach is a context switch. Learn these until they're automatic.

- Cmd/Ctrl+P — Quick file open
- Cmd/Ctrl+Shift+P — Command palette
- F12 / Cmd+Click — Go to definition
- Shift+F12 — Find all references
- Cmd/Ctrl+D — Select next occurrence
- Cmd/Ctrl+Shift+L — Select all occurrences
- Alt+Up/Down — Move line
- Cmd/Ctrl+/ — Toggle comment
- Cmd/Ctrl+B — Toggle sidebar
- Cmd/Ctrl+` — Toggle terminal

### Multi-Cursor & Selection

WHY: Multi-cursor editing replaces dozens of find-replace operations. Once you see someone use it well, you can't go back.

- Alt+Click — Add cursor
- Cmd/Ctrl+Alt+Up/Down — Add cursor above/below
- Select word → Cmd/Ctrl+D repeatedly
- Box selection (Alt+Shift+drag)
- Expand/shrink selection
- Select all in scope

### Code Navigation

WHY: Large codebases require fast navigation. These features let you understand code without scrolling through files.

- Go to symbol in file (Cmd/Ctrl+Shift+O)
- Go to symbol in workspace
- Peek definition (Alt+F12)
- Navigate back/forward
- Breadcrumbs navigation
- Outline view

### Refactoring & Code Actions

WHY: Manual renaming causes bugs. IDE refactoring tools update all references safely. Extract operations clean up code without risk.

- Rename symbol (F2)
- Extract method/variable
- Inline variable
- Quick fixes (Cmd/Ctrl+.)
- Organize imports
- Format on save

### The Debugger

WHY: Print-statement debugging is slow and pollutes code. A proper debugger lets you inspect state, step through code, and test hypotheses interactively.

- Setting breakpoints
- Conditional breakpoints
- Step over/into/out
- Watch expressions
- Call stack inspection
- Debug console evaluation

## Hands-on Projects

### Mouseless Day

Spend one full day coding without touching your mouse. Put it in a drawer. When you need to do something, find the keyboard shortcut. Write down every shortcut you learn. Repeat weekly.

Keyboard shortcuts | Flow state | Muscle memory

### Debug a Complex Bug

Take a bug you've been print-debugging and solve it using only the debugger. Set breakpoints, inspect variables, evaluate expressions. No console.log or print() allowed.

Debugging | Breakpoints | State inspection

## Resources

REFERENCE **VS Code Keyboard Shortcuts PDF**

TOOL **Key Promoter X (JetBrains plugin)**

DOCUMENTATION **VS Code Tips and Tricks**

**Milestone**

Go 30 minutes without touching the mouse while coding. Use multi-cursor to make a repetitive edit. Debug a bug using breakpoints instead of print statements. Have at least 10 shortcuts in muscle memory.

# 4 HTTP & API Tools

Intensive: Week 4 | Balanced: Weeks 7-8 | Flexible: Weeks 13-16

Every web developer works with APIs. Understanding HTTP deeply — and having tools to inspect and test it — eliminates entire categories of bugs. You'll stop guessing why requests fail and start knowing. This phase covers the tools that let you see exactly what's happening on the wire.

## Learning Objectives

- Understand HTTP methods, headers, and status codes
- Test APIs quickly from terminal with curl
- Use Postman/Insomnia for complex API workflows
- Debug requests using browser DevTools
- Understand common authentication patterns

## Topics to Cover

### HTTP Fundamentals

WHY: HTTP is the language of the web. Understanding it means understanding why your API returns 401 vs 403, why CORS errors happen, and what caching headers actually do.

- Request/Response anatomy
- Methods (GET, POST, PUT, DELETE, PATCH)
- Status codes (2xx, 3xx, 4xx, 5xx)
- Headers that matter (Content-Type, Auth, Cache)
- Query params vs body
- HTTPS and TLS basics

### curl Mastery

WHY: curl is everywhere. It's on every server, in every Stack Overflow answer, in every API doc. Learning curl means you can test any API anywhere.

- Basic GET request
- POST with JSON body (-d, -H)
- Authentication (-u, -H Authorization)
- Following redirects (-L)
- Verbose output (-v)
- Saving/loading from files
- Copying requests from DevTools

### Postman / Insomnia

WHY: GUI tools make API exploration faster. Collections save test cases. Environment variables prevent mistakes. They're better for complex workflows.

- Creating requests
- Environment variables
- Collections and folders
- Pre-request scripts
- Tests and assertions
- Sharing with team

### Browser DevTools Network Tab

WHY: When your app misbehaves, the Network tab shows exactly what happened. You can replay requests, copy as curl, throttle speed, and see timing breakdowns.

- Filtering requests
- Inspecting headers/body
- Copy as curl/fetch
- Timing breakdown (TTFB, download)
- Throttling simulation
- Blocking requests
- WebSocket inspection

## Hands-on Projects

### API Test Suite in Postman

Pick a public API (GitHub, Spotify, Weather). Build a Postman collection with at least 10 requests covering different endpoints and methods. Add environment variables for API keys. Write tests to verify responses.

Postman | API testing | Environments

### curl Scripting

Write a bash script that uses curl to: authenticate with an API, fetch data, parse JSON with jq, and output formatted results. Make it reusable with command-line arguments.

curl | JSON parsing | Bash scripting

## Resources

REFERENCE **HTTP Status Codes**

TUTORIAL **curl Cookbook**

DOCUMENTATION **Postman Learning Center**

### Milestone

Debug an API issue using only DevTools (no backend access). Write a curl request from memory for authenticated POST with JSON. Maintain a Postman collection for an API you use.

# 5 Browser DevTools Deep Dive

Intensive: Week 5 | Balanced: Weeks 9-10 | Flexible: Weeks 17-20

DevTools is the most powerful debugging tool for web developers, yet most people only use console.log. The Performance tab finds bottlenecks. The Memory tab finds leaks. The Network tab shows exactly what's happening. This phase turns DevTools from 'that thing with console.log' into your primary debugging weapon.

## Learning Objectives

- Debug JavaScript effectively using Sources panel
- Profile performance and find bottlenecks
- Identify and fix memory leaks
- Debug CSS layout issues
- Use DevTools to understand third-party code

## Topics to Cover

### Console Beyond log()

WHY: console.log is just the beginning. console.table, console.time, console.trace, and console.group make debugging output actually useful instead of walls of text.

- console.table() for arrays/objects
- console.time()/timeEnd() for timing
- console.trace() for stack traces
- console.group()/groupEnd()
- console.assert()
- $0, $_, $() in console

### Sources & Debugging

WHY: Setting breakpoints in DevTools beats sprinkling console.logs. You can pause on exceptions, edit code live, and step through async code.

- Breakpoints (line, conditional, DOM)
- Pause on exceptions
- Blackboxing third-party scripts
- Local overrides (edit production)
- Workspaces (persist changes)
- Async debugging

### Performance Profiling

WHY: 'It feels slow' isn't actionable. The Performance tab shows exactly where time goes. You'll see if it's JavaScript, rendering, network, or layout that's slow.

- Recording a performance trace
- Reading flame graphs
- Main thread vs other threads
- Layout thrashing detection
- Long tasks identification
- Core Web Vitals (LCP, CLS, INP)

### Memory Analysis

WHY: Memory leaks kill user experience over time. Heap snapshots show what's consuming memory. Comparison shows what's growing. This is how you find leaks.

- Heap snapshots
- Comparing snapshots
- Allocation timeline
- Detached DOM nodes
- Closure-caused leaks
- Finding retained objects

### Elements & CSS Debugging

WHY: 'Why isn't this CSS working?' has an answer in DevTools. Computed styles show what actually applies. The box model visualizes spacing issues.

- Computed styles panel
- CSS specificity debugging
- Box model visualization
- Flexbox/Grid inspectors
- Forcing element states (:hover, :focus)
- Coverage tab (unused CSS)

## Hands-on Projects

### Performance Audit

Take a slow webpage (or build one intentionally). Use the Performance tab to identify the bottleneck. Is it JavaScript? Layout? Network? Fix the issue and measure the improvement.

Performance profiling | Flame graphs | Optimization

### Memory Leak Detective

Create a page with an intentional memory leak (event listener not cleaned up, DOM reference held). Use heap snapshots to identify the leak and fix it.

Heap snapshots | Memory analysis | Debugging

## Resources

DOCUMENTATION **Chrome DevTools Docs**

COURSE **Mastering DevTools (Frontend Masters)**

TIPS **DevTools Tips**

### Milestone

Profile a real performance issue using the Performance tab. Find and fix a memory leak using heap snapshots. Use local overrides to test a fix on production without deploying.

# 6 Docker Essentials

Intensive: Week 6 | Balanced: Weeks 11-12 | Flexible: Weeks 21-26

Docker solves 'works on my machine.' It also makes setting up complex environments trivial — databases, message queues, entire stacks with one command. You don't need to become a DevOps expert, but understanding containers is now a baseline skill for all developers.

## Learning Objectives

- Understand what containers are and why they matter
- Write Dockerfiles for your applications
- Use docker-compose for multi-container setups
- Debug containerized applications
- Optimize images for size and build time

## Topics to Cover

### Container Concepts

WHY: Understanding what containers actually are (process isolation, not VMs) helps you use them correctly and debug when things go wrong.

- Containers vs VMs
- Images vs containers
- Layers and caching
- Registries (Docker Hub)
- Container lifecycle
- Why containers for dev?

### Docker CLI Essentials

WHY: These commands cover 90% of daily Docker usage. Learn them well before moving to docker-compose.

- docker run (ports, volumes, env vars)
- docker ps (list containers)
- docker logs (view output)
- docker exec (run commands inside)
- docker build (create images)
- docker stop/rm (cleanup)

### Writing Dockerfiles

WHY: The Dockerfile is your application's recipe. Understanding layers, caching, and multi-stage builds makes images smaller and builds faster.

- FROM, RUN, COPY, CMD
- Layer caching (order matters)
- Multi-stage builds
- .dockerignore
- Build arguments and env vars
- Health checks

### Docker Compose

WHY: Real applications have dependencies — databases, caches, queues. Compose lets you define entire stacks that come up with one command.

- docker-compose.yml structure
- Services, networks, volumes
- Environment variables
- Dependency ordering
- Development vs production configs
- Common patterns (db, cache, app)

## Hands-on Projects

### Containerize an Application

Take one of your existing projects. Write a Dockerfile for it. Write a docker-compose.yml that includes the app and its database. Someone should be able to clone your repo and run 'docker compose up' to have everything working.

Dockerfile | docker-compose | Environment config

### Local Dev Stack

Create a docker-compose.yml with services you commonly need: PostgreSQL, Redis, maybe Elasticsearch or RabbitMQ. Add healthchecks. Use it as your go-to development environment.

docker-compose | Service configuration | Networking

## Resources

TUTORIAL **Docker Curriculum**

DOCUMENTATION **Docker Documentation**

GUIDE **Dockerfile Best Practices**

### Milestone

Run 'docker compose up' and have your entire dev stack running. Write a Dockerfile that uses multi-stage builds. Debug a containerized app using docker logs and docker exec.

# 7 AI Coding Assistants

Intensive: Week 7 | Balanced: Weeks 13-14 | Flexible: Weeks 27-30

76% of developers are using or planning to use AI coding tools. They're not going away. This phase teaches you to use them effectively — as power tools, not crutches. The goal is to be faster without becoming dependent or losing understanding of your own code.

## Learning Objectives

- Use GitHub Copilot effectively for common patterns
- Write prompts that get useful results
- Know when AI helps vs when it hurts
- Review AI-generated code critically
- Use AI for learning, not just output

## Topics to Cover

### Effective Prompting for Code

WHY: AI assistants are only as good as your prompts. Clear context and specific asks get useful code. Vague requests get generic garbage.

- Context matters (comments, function names)
- Breaking down complex requests
- Asking for explanations, not just code
- Iterating on responses
- Specifying constraints and patterns
- When to start fresh vs iterate

### GitHub Copilot Patterns

WHY: Copilot excels at certain tasks. Knowing where it shines (boilerplate, tests, common patterns) vs where it fails (complex logic, novel algorithms) makes you faster.

- Tab completion flow
- Comment-driven development
- Generating tests from functions
- Documentation generation
- Exploring APIs
- Learning new languages/frameworks

### Critical Review of AI Code

WHY: AI generates plausible code that can be wrong. Security vulnerabilities, subtle bugs, outdated patterns — you need to review AI code more carefully, not less.

- Security red flags
- Logic verification
- Checking edge cases
- Performance implications
- Style consistency
- Dependency choices

### AI as Learning Tool

WHY: The best use of AI isn't generating code — it's explaining code. Use it to understand unfamiliar codebases, learn new concepts, and explore alternatives.

- Explaining existing code
- Asking 'why' questions
- Exploring alternative approaches
- Understanding error messages
- Learning patterns and idioms
- Code review assistance

## Hands-on Projects

### AI-Assisted Feature Implementation

Implement a feature using AI assistance. Document: what prompts worked well, what you had to fix, where AI saved time, where it wasted time. Reflect on when to use it.

Prompting | Critical review | Self-reflection

### Code Explanation Challenge

Find a complex open-source function you don't understand. Use AI to explain it. Then verify the explanation by reading the actual code and tests. Note where AI was right vs wrong.

Learning | Verification | Understanding

## Resources

DOCUMENTATION **GitHub Copilot Documentation**

GUIDE **Prompt Engineering Guide**

### Milestone

Use AI assistance for a full day of coding. Track when it helped vs hindered. You should be faster overall, but able to explain every line of generated code. Never merge AI code you don't understand.

# Your Path Forward

### Career Opportunities

- ✓ More productive in any role
- ✓ Faster debugging
- ✓ Better collaboration (clean Git history)
- ✓ Smoother onboarding to new projects
- ✓ Confidence with unfamiliar codebases

### Tips for Success

- Learn one shortcut per day — don't try to memorize everything at once
- Put your mouse in a drawer for an hour. You'll learn shortcuts fast.
- Install Key Promoter X (JetBrains) or learn by watching faster developers
- Build muscle memory through repetition, not memorization
- When you look something up twice, make an alias or snippet
- Don't customize everything at once — start minimal and add as needed
- Watch senior developers work — you'll learn more in an hour than a week of tutorials
- These skills compound. Invest early.
- Type 'tldr <command>' before reading man pages
- Use AI to explain, not just generate

### Next Steps

1. Install a modern terminal (iTerm2, Warp, Windows Terminal)
2. Switch to Zsh with Oh My Zsh if you haven't
3. Install fzf and ripgrep right now
4. Print a keyboard shortcut cheat sheet and keep it visible
5. Pick ONE phase and focus on it for a week
6. Find someone faster than you and watch them work

## Ready to Start Your Journey?

Everything in this roadmap is free to learn on your own. But if you'd like structured guidance, hands-on projects, and mentor support — we'd love to have you at Crio.Do

Explore Crio.Do

https://crio.do